回顾：Transformer 的数据怎样流动？

从一句话出发，沿着分词、向量、注意力和输出，走完一次预测。

整条路径：预测下一个 token

01	文本 → Tokenizer	IDs [B,T]
02	Embedding 查表	X [B,T,D]
03	Decoder Block × L	H [B,T,D]
04	Norm + LM Head	logits [B,T,V]
05	最后位置的 Softmax	选择下一 ID

把新 ID 追加到序列，再预测下一项。

单层 Block：混合上下文，再加工特征



每层输入、输出都保持 [B,T,D]。

B 句子数　T 序列长度　D 向量维度　V 词表大小

文本变 ID，向量穿过多层 Block，最后得到下一 token 的概率。

从上节课的计算，走到今天的输入端

把上节课的人工算例，接到真实模型的三个输入接口。

上节课的人工算例：“我 / 喜欢 / 喝” → 下一 token

Q/K 匹配 → Softmax 权重 ≈ [0.248,0.248,0.503] → 加权混合 V

混合向量 ≈ [0.752,0.752] → 输出映射 → 酒 / 牛奶的概率 ≈ [0.680,0.320]

01 Tokenizer

文本 → IDs [B,T]

上节课：按空格切分。

今天：BPE 怎样学规则、怎样执行？

02 Embedding

ID → 向量 [B,T,D]

上节课：人工二维向量。

今天：怎样查表、怎样通过训练更新？

03 位置编码

位置 t → 注意力的位置关系

上节课：认识顺序的作用。

今天：输入加 PE，或在 Q/K 上用 RoPE。

今天的目标：说明每个接口为什么这样设计，并在代码里找到对应运算。

本课聚焦：Tokenizer、Embedding、位置编码在整体架构中的作用。

BPE：先统计，再合并

每轮重新统计当前相邻片段，选择最高频的一对。

① 语料与权重：不是每句话只算一次

我喜欢喝奶茶 × 5

你喜欢喝奶茶 × 3

我喜欢喝咖啡 × 2

喜+欢：5+3+2=10；我+喜：5+2=7

② 每次合并后重算；同频按首次出现处理

rank 0：喜 + 欢　频次 10

rank 1：喜欢 + 喝　频次 10

rank 2：喜欢喝 + 奶　频次 8

rank 3：喜欢喝奶 + 茶　频次 8

先从字符出发，学出“喜欢喝奶茶”；这是高频片段，不是人工指定的语言学词语。

```py
counts[pair] += corpus[text]
pair = max(counts, key=counts.get)
```

实验 01 · bpe_core.py

频次来自语料；合并后要重新统计。

BPE 编码：谁的 rank 更小？

规则已经冻结；每次在当前候选中选择优先级最高的一对。

① 冻结规则：不再计算语料频次

0：喜 + 欢
1：喜欢 + 喝
2：喜欢喝 + 奶
3：喜欢喝奶 + 茶

② 编码“你喜欢喝奶茶”：按最小可用 rank

你 / 喜 / 欢 / 喝 / 奶 / 茶

0 → 你 / 喜欢 / 喝 / 奶 / 茶
1 → 你 / 喜欢喝 / 奶 / 茶
2 → 你 / 喜欢喝奶 / 茶
3 → 你 / 喜欢喝奶茶

重叠反例：bc 的 rank=0、ab 的 rank=1；abc 先合并 bc → a/bc，而非 ab/c。

```py
rank, pair = min(candidates, key=lambda item: item[0])
parts = merge_pair(parts, pair)
```

实验 02 · bpe_core.py

merge rank 决定合并顺序，token ID 决定查表地址。

把 BPE 接到真实文本上

ByteLevel 处理字节与边界；BPE 处理合并规则。

字节、边界、合并：各自负责什么？

字符演示说明合并规则；真实库还要处理文本边界、基础字节和解码。

本课 ByteLevel 用完整256字节字母表覆盖UTF-8字节，并设置4个特殊符号。

保存的不只是词表，还包括merges、预切分和decoder；之后加载完整配置。

对应分词器的五个步骤

```
Tokenizer(models.BPE(...))
pre_tokenizers.ByteLevel(...)
trainers.BpeTrainer(...)
tokenizer.train_from_iterator(...)
tokenizer.save(...)
```

PY

```
tokenizer = load_tokenizer()
enc = tokenizer.encode(text)
```

实验 03 · train_tokenizer.py

先训练并保存配置，再加载同一配置进行编码。

ID 必须与参数表配套

同样大小的词表，也可能把同一片段映射到不同的地址。

真实配置：普通文本的 ID 只对当前词表有意义

我喜欢喝牛奶。 → [300,304,262] → E[300]、E[304]、E[262]

地址相同、含义不同：假设两份词表

词表 A：奶茶 → 7

词表 B：咖啡 → 7

只换词表会怎样?

模型原先把 E[7] 当作奶茶的表示。只替换词表后，咖啡会读到原来的参数行。

词表配置与参数表，必须成套加载。

默认特殊符号：pad=0，unk=1，bos=2，eos=3；本课显式添加 bos/eos。

实验 04 · runtime.py

更换 tokenizer 时，必须核对 ID 与模型参数的对应关系。

Embedding：一个 ID 选择一行

查表与 one-hot 乘法前向等价；实际实现直接选行。

$$\mathbf{X} = \mathbf{E}[I] = \text{one_hot}(I)\,\mathbf{E}$$

机制：每个量在做什么

查表等于用one-hot选择一行，既保留可学习参数，又避免显式分配大one-hot张量。

算术小表中，ID=[1,2,1]会读到第1行、第2行、再读一次第1行。

算例：跟着数值走一遍

算术小表 E 的第 1、2 行：

E[1] = [0.2, -0.1, 0.4]

E[2] = [0.5, 0.3, -0.2]

I=[1,2,1] → 依次取第1、第2、第1行

PY
```
vectors = embedding(ids)
one_hot @ embedding.weight
```

实验 05 · embeddings.py

ID 是地址；可训练的表示存放在 E 的每一行。

不同长度，怎样组成 batch?

先添加起止标记，再补齐；pad 只是占位。

B=2，T=6：t1…t4 是普通 token；起止标记也占位置

句1	bos	t1	t2	t3	eos	pad
句2	bos	t1	t2	t3	t4	eos

三个接口分别负责什么?

padding_idx	抑制 pad 行的 lookup 梯度
padding key mask	Attention 不读取 pad key
ignore_index	损失不计算 pad 标签

实验 05 · runtime.py

padding_idx、注意力 mask 和损失忽略，需要分别处理。

读了同一行，梯度就要相加

共享参数接收所有读取位置的贡献；这里先看 lookup 路径。

$$\frac{\partial L}{\partial \mathbf{E}_j} = \sum_{(b,t)\,:\,I_{b,t}=j} \frac{\partial L}{\partial \mathbf{X}_{b,t}} \qquad (j \neq \text{pad})$$

机制：每个量在做什么

把同一行看成被多处引用的共享参数；每个位置贡献自己的向量梯度。

若ID1两次上游梯度为[1,0]和[0,2]，第1行收到[1,2]，不是取平均也不是覆盖。

算例：跟着数值走一遍

ID 1：[1,0]+[0,2] = [1, 2]

ID 2：第2行梯度 = [3, 4]

pad 行的 lookup 梯度为 [0,0]。上游梯度是向量，累加不是数出现次数这么简单。

PY

```
loss = (embedding(ids) * upstream).sum()
loss.backward()
```

实验 06 · embeddings.py

反向是 scatter-add：累加贡献，而不是覆盖旧值。

预测误差怎样传回向量?

单位置交叉熵：先得到类别误差，再乘回输出权重。

$$\frac{\partial L}{\partial \mathbf{e}} = (\mathbf{p} - \text{one_hot}(y))\mathbf{W}_{\text{out}}$$

机制：每个量在做什么

类别编号0/1/2，目标y=1，预测p=[0.1,0.7,0.2]；交叉熵为-ln(0.7)≈0.3567。

类别误差p-one_hot(1)=[0.1,-0.3,0.2]，右乘输出权重得到输入向量梯度。

算例：跟着数值走一遍

p-one_hot(y) = [0.1, -0.3, 0.2]

W_out 三行：[1,0]、[0,1]、[-1,0.5]

向量梯度 = [-0.1, -0.2]

目标类的概率偏低，误差经 W_out 传回。

```py
loss = F.cross_entropy(logits, target)
loss.backward()
```

实验 07 · gradient_chain.py

先算每次出现的向量梯度，再按 ID 汇总到参数表。

让 Embedding 真正学一次

57 对相邻 token； Embedding → Linear → CrossEntropy。

$$L = -\frac{1}{N}\sum_{r=1}^{N}\log p(y_r \mid x_r)$$

机制：每个量在做什么

把相邻token构成(x,y)：当前ID查表，经Linear产生所有候选的logits，再与真实下一ID比较。

每步依次前向、清梯度、backward、optimizer.step；梯度负责指出如何调整参数。

算例：跟着数值走一遍

步数	0	30	120
loss	5.7565	0.3607	0.3559

26 行参数改变；模型只看当前 token。同一输入存在多种后续，loss 不必归零。

```py
logits = output(embedding(x)); loss = F.cross_entropy(logits, y)
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

实验 08 · embedding_train.py

训练让“当前 token 的向量”逐渐有助于预测下一项。

一张表，两条梯度路径

权重绑定：输入查表和输出分类器共享同一个 E。

$$\mathbf{z} = \mathbf{h}\mathbf{E}^T + \mathbf{b}, \qquad \nabla_{\mathbf{E}} L = \nabla_{\mathbf{E}}^{\text{lookup}} L + \nabla_{\mathbf{E}}^{\text{output}} L$$

机制：每个量在做什么

不绑定时，输入E与输出W是两个参数；绑定后输出直接使用E的转置。

输入查表只读取少数行，输出softmax却把全词表作为候选，因此更多行可能收到梯度。

算例：跟着数值走一遍

输入路径：只读取当前出现的行。

输出路径：softmax 对全词表候选求导。

两条贡献相加：没出现在输入里的行，也可能因输出分类收到梯度。

PY

```
head.weight = tied_embedding.weight
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

课后扩展 · tools/check_experiments.py

绑定后，没作为输入的行也可能收到输出梯度。

没有位置，重排会怎样?

假设：没有位置机制，也没有因果 mask。

$$\mathrm{Attn}(\Pi \mathbf{X}) = \Pi \mathrm{Attn}(\mathbf{X})$$

机制：每个量在做什么

“我/喜欢/你”换成“你/喜欢/我”，仅查表得到的仍是同一组内容向量。

无位置、无mask时，Q/K/V随行重排，scores行列同时重排，输出也对应重排。

算例：跟着数值走一遍

内容顺序：我 / 喜欢 / 你

重排为：你 / 喜欢 / 我

Q/K/V 都随行重排，scores 的行列一起重排。加上固定位置坐标后，一般不再满足这个等式。

PY

```
rearranged = attention(x[permutation])
torch.allclose(rearranged, original[permutation])
```

实验 09 · position_mechanics.py

只按内容匹配时，输出跟着输入一起重排。

把位置向量加进输入

内容与位置使用同样的维度，相加后形状保持不变。

$$\mathbf{X}_{b,t} = \mathbf{E}[I_{b,t}] + \mathrm{PE}(t)$$

机制：每个量在做什么

内容向量表示“是什么”，位置向量提供“在第几个token”；相加后仍是D维。

学习式位置表通过训练更新，直接查询范围受表长L限制；正弦编码由固定函数生成。

算例：跟着数值走一遍

D=4，位置 t=0 的正弦向量

内容：[0.2, -0.1, 0.4, 0]

位置：[0, 1, 0, 1]

相加：[0.2, 0.9, 0.4, 1]

PY

```
pe = sinusoidal(torch.arange(3), 4)
absolute_original = attention(x + pe)
```

实验 10 · positions.py

位置是 token 序号，不是原文的字符 offset。

位置 × 频率，得到角度

不同维度对使用不同频率，再交错存放 sin 与 cos。

$$\omega_i = 10000^{-2i/D}, \quad \theta_{t,i} = t\omega_i$$

$$\mathrm{PE}(t, 2i) = \sin\theta_{t,i}, \quad \mathrm{PE}(t, 2i+1) = \cos\theta_{t,i}$$

机制：每个量在做什么

D=4：两对维度的频率为 [1,0.01]。t=1 时角度为 [1,0.01]。

[T,1] × [1,D/2] 广播得到 [T,D/2]；不是逐个 token 写循环。

算例：跟着数值走一遍

PE(0) = [0, 1, 0, 1]

PE(1) ≈ [0.8415, 0.5403, 0.0100, 1.0000]

高低频率提供不同的位置变化尺度。

PY

```
angles = positions[:, None] * frequencies[None, :]
result[:, 0::2], result[:, 1::2] = angles.sin(), angles.cos()
```

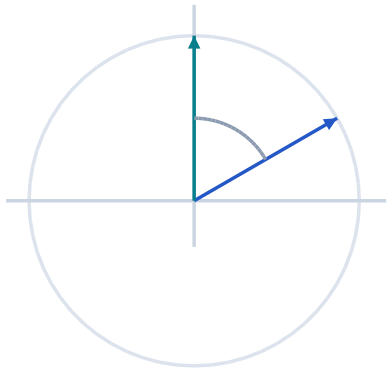
实验 10 · positions.py

positions → angles → sin / cos：每一步都有明确形状。

RoPE：位置变成箭头的转角

先看一对二维分量：每前进一个位置，箭头就多转一点。

① 位置 1 和 3



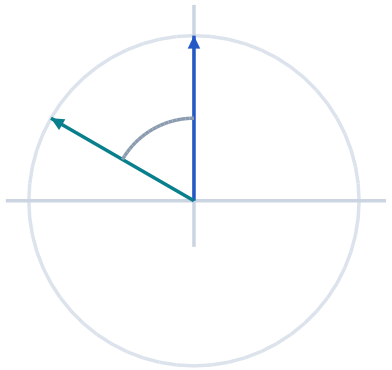
Q：位置 1 → 30°

K：位置 3 → 90°

夹角仍为 60°

位置相差 2 格

② 两个位置都向后移 2 格



Q：位置 3 → 90°

K：位置 5 → 150°

夹角仍为 60°

位置相差 2 格

单位箭头的点积 = 夹角的余弦

$$\cos((3-1)\times 30^\circ) = \cos 60^\circ = 0.5$$

图解设定：Q、K 起初都为 [1,0]，每格旋转 30°。实验 11：rotate_pair

两个箭头一起转，夹角不变；位置差因此进入注意力分数。

从两个箭头，到整个 Q / K 张量

每两个分量组成一支箭头，各对维度按不同速度旋转。

高维向量：拆成多支二维箭头

[x0, x1 | x2, x3 | x4, x5 | ⋯]

第 i 对的角度：θ = 位置 t × 频率 ωi。

例如 d_h=4：两对维度的频率为 1、0.01；同一位置，一对转得快，一对转得慢。

Q 和 K 都这样转，再做点积；V 保留原来的内容向量。

一支箭头怎样转？公式逐项对应代码

$$x'_{2i} = x_{2i}\cos\theta - x_{2i+1}\sin\theta$$

$$x'_{2i+1} = x_{2i}\sin\theta + x_{2i+1}\cos\theta$$

原来的两个坐标 → 旋转后的两个坐标

[1,0] 转 30° → [0.8660, 0.5000]

```py
result[..., 0::2] = even * cos - odd * sin
result[..., 1::2] = even * sin + odd * cos
```

实验 11 · positions.py

RoPE 旋转 Q 和 K；位置通过它们的点积进入 Attention。

位置关系与可读范围

RoPE 改变分数；mask 决定哪些 key 可以被读取。

$$\mathbf{A} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{T}}{\sqrt{d_h}} + \mathbf{M}\right)$$

机制：每个量在做什么

位置机制改变分数的数值关系，因果mask禁止读取未来，padding key mask禁止读取补齐项。

禁止位置设为-∞，softmax后权重为0；示意中最后一个key为pad，整列权重为0。

S全0、末列为pad：注意力权重

1	0	0	0
1/2	1/2	0	0
1/3	1/3	1/3	0
1/3	1/3	1/3	0

pad query 的输出另行清零。

PY
```
forbidden = future | ~valid[None, :]
weights = scores.masked_fill(forbidden, -torch.inf).softmax(-1)
```

实验 12 · input_pipeline.py

有了位置机制，仍要屏蔽未来 key 与 padding key。

沿真实输入，把形状接起来

本地 tokenizer → Embedding → 多头 Q/K → RoPE + mask。

从本地 ID 到多头注意力输出：每个轴都有含义

[2,6] → [2,6,8] → Q/K [2,2,6,4] → scores [2,2,6,6] → [2,6,8]

对应实现与每步目的

1 查表	embedding(ids)	离散地址 → D维参数
2 投影、拆头	Linear → view → transpose	D=8 拆成 H=2、d_h=4
3 位置、遮罩	apply_rope → masked_fill	位置进入Q/K，mask限制key
4 混合、合头	softmax → @v → reshape	沿key归一化，再合并各head

实验 12 · input_pipeline.py

把分词、向量、旋转与 mask 接起来，完成一次注意力计算。

用三个问题，检查是否理解

解释一次编码、一次反传、一次位置计算。

先独立回答，再用下面的机制核对

编码 为什么abc不是最长前缀ab/c?

核对：可合并的当前相邻候选中，选最小rank。

梯度 同一ID出现三次，第j行怎样更新?

核对：把三个位置的上游向量梯度相加。

位置 RoPE已有相对位移，mask还做什么?

核对：位置参与打分，mask限制哪些key可读。

课后：改词表、查梯度、验证旋转，并保留实验结果。